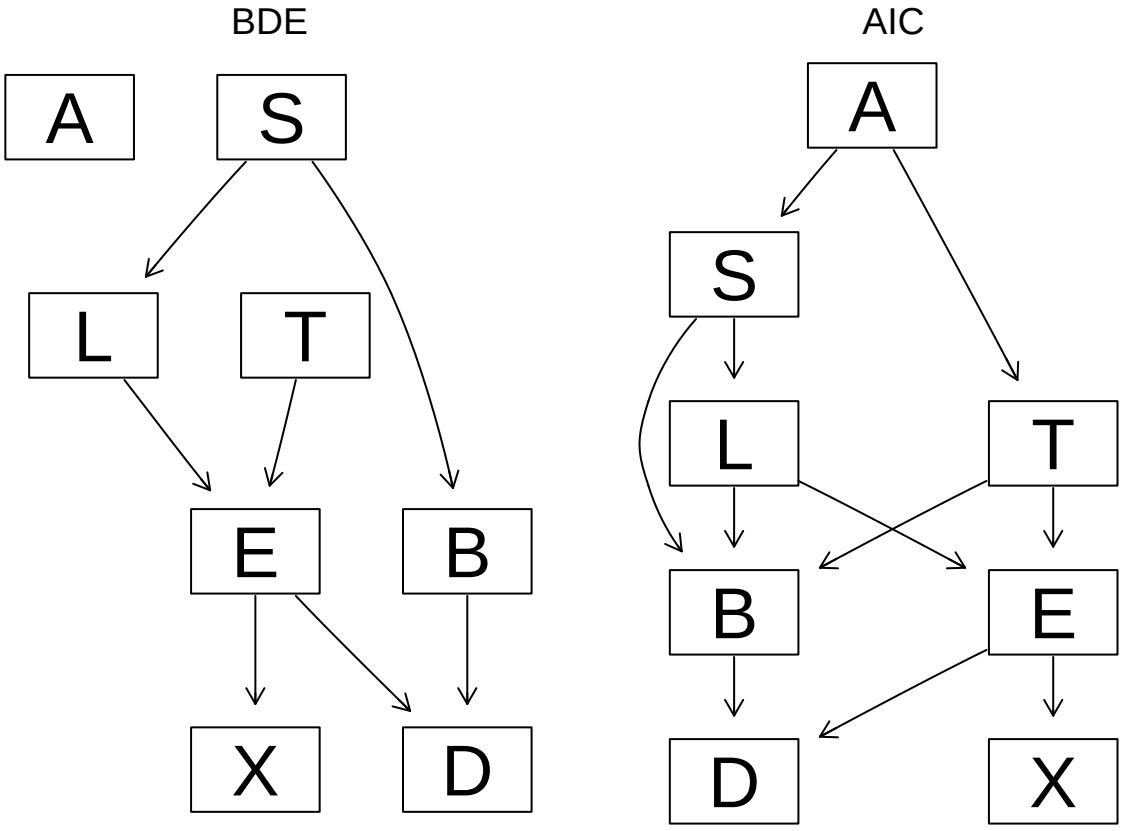
Lab 1: Bayesian Networks

Linus Göransson - lingo978

2026-09-14

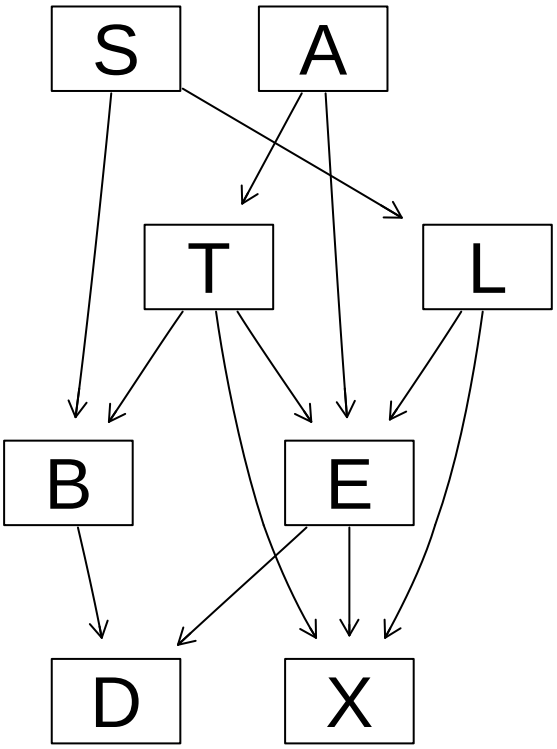
Question 1

Show that multiple runs of the hill-climbing algorithm can return non-equivalent Bayesian network (BN) structures. Explain why this happens.

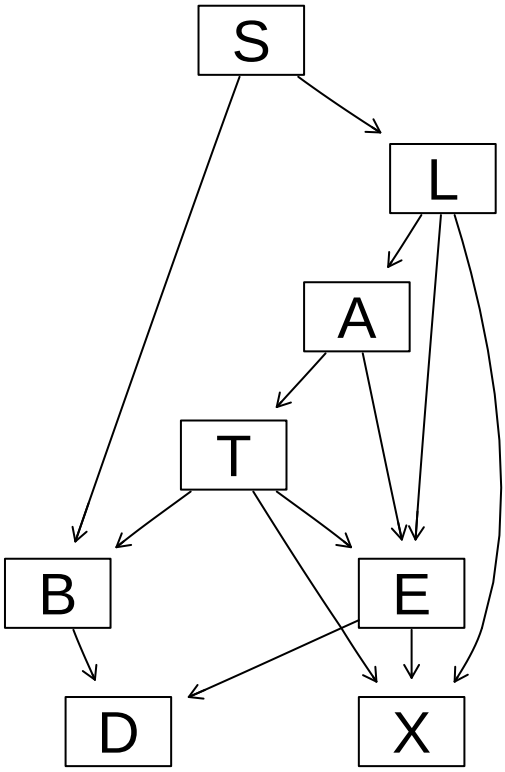



```
## [1] "Different number of directed/undirected arcs"
```

ISS = 5

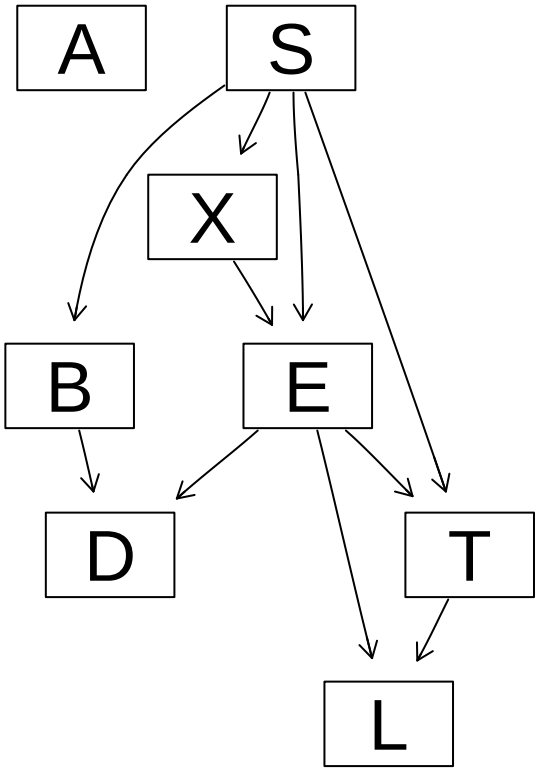


ISS = 10

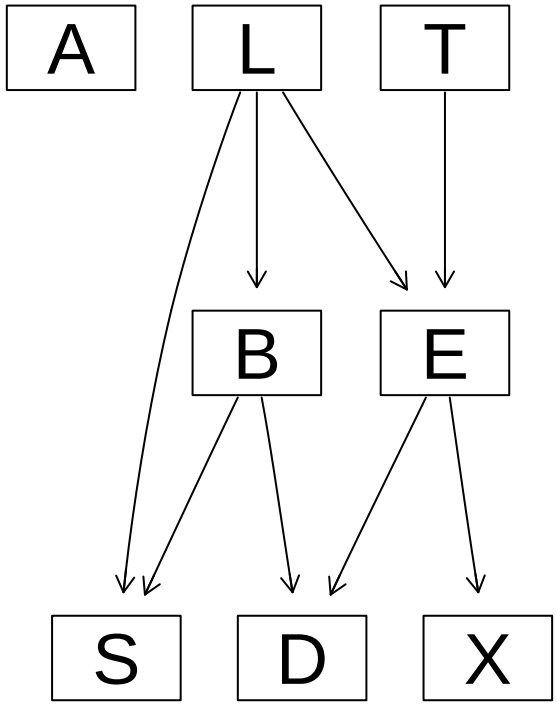



```
## [1] "Different number of directed/undirected arcs"
```

Random DAG 1



Random DAG 2



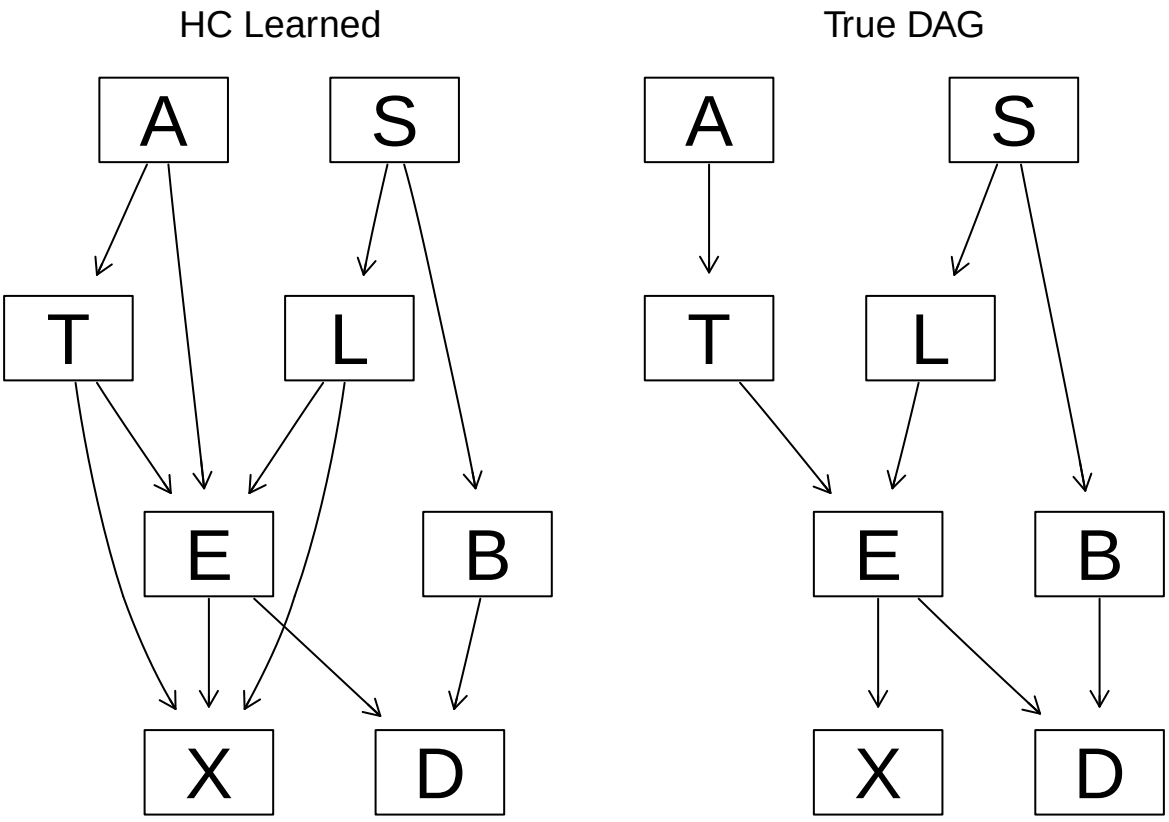
```
## [1] "Different number of directed/undirected arcs"
```

Answer. All three variants of hill climbing (different score functions, different imaginary sample sizes, and different random starting structures; code in Appendix A.1) return structures that are not equivalent. This is shown by the plots and by converting the DAGs to equivalence classes by using the `cpdag()` function and comparing them using `all.equal()`, meaning the two DAGs do not belong to the same equivalence class.

Hill climbing is a greedy local search that repeatedly accepts the single edge change giving the largest improvement in the score. Because the search is greedy, it can get stuck in different local optima, so the returned structure is not guaranteed to be the global optimum.

Question 2

Learn a BN from 80 % of the Asia dataset. Use the BN learned to classify the remaining 20 % of the Asia dataset in two classes: S = yes and S = no. In other words, compute the posterior probability distribution of S for each case and classify it in the most likely class.




```
##      hc_model_preds
##        no yes
##   no  337 176
##   yes 121 366
```

```
##      true_model_preds
##        no yes
##   no  337 176
##   yes 121 366
```

Answer. A BN was learned from the 80 % training split using hill climbing with the BDE score and fitted with MLE (Appendix A.2). As the plots and the two confusion matrices show, classification based on the learned network and based on the true network gives identical results. The reason is that when every node is

observed, the posterior of S depends only on S's Markov blanket, and both the learned and the true DAG share the same local structure around S (nodes L and B). Although some edge configurations (around T, E, X) differ, those nodes are conditionally independent of S given its Markov blanket and have no effect on the prediction of S.

Question 3

Now, you are asked to classify S given observations only for the so-called Markov blanket of S

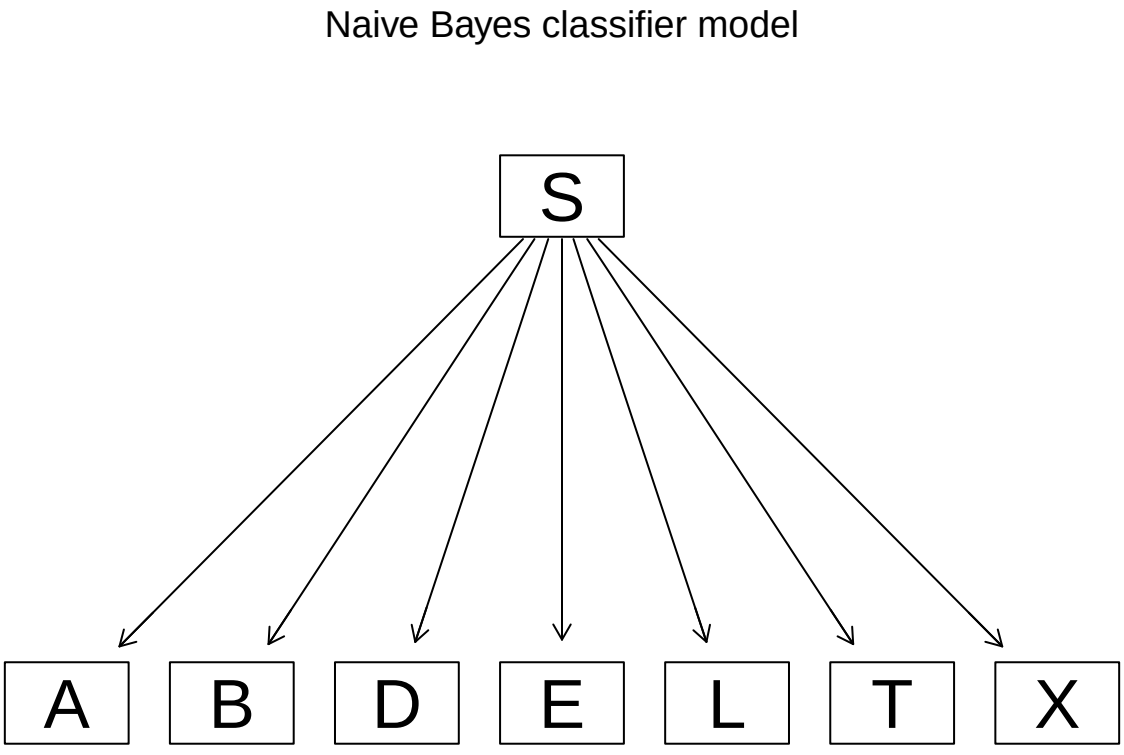
```
##      hc_mb_preds
##        no yes
##   no  337 176
##   yes 121 366
```

```
##      true_mb_preds
##        no yes
##   no  337 176
##   yes 121 366
```

Answer. Using only S's Markov blanket as evidence (Appendix A.3) produces the same confusion matrices as in Exercise 2. This is expected because the learned DAG and the true DAG share the same Markov blanket for S (nodes L and B), and evidence outside the Markov blanket does not change the posterior probability of S.

Question 4

Repeat the exercise (2) using a naive Bayes classifier




```
##      nbc_preds
##        no yes
##   no  359 154
```

```
##   yes 180 307
```

Answer. The naive Bayes network assumes all features are conditionally independent given S (Appendix A.4). The resulting confusion matrix differs from Exercise 2, because the naive Bayes assumption removes the dependencies that exist between the features (e.g. $T \to E$ and $B \to D$) in the true and the learned network, leading to different posterior estimates of S.

Question 5

Explain why you obtain the same or different results in the exercises (2-4).

Answer. Exercises 2 and 3 produce the same results because both the learned HC DAG and the true DAG share the same Markov blanket of S. By the conditional independence property of BNs, $P(S \mid \text{evidence})$ depends only on the evidence in S's Markov blanket.

Exercise 4 differs because the naive Bayes classifier assumes all features are conditionally independent fiven given S, removing edges like $T \to E$ and $B \to D$, ignoring dependencies among the features that are present in the data, which leads to worse predictive performance.

Appendix A: R Code

A.1 Question 1: Hill climbing with variations

```r
# Learn structure using HC with different score
dag1 <- hc(asia, score = "bde")
dag2 <- hc(asia, score = "aic")
par(mfrow = c(1, 2), mar = c(1, 1, 2, 1))
graphviz.plot(dag1, main = "BDE")
graphviz.plot(dag2, main = "AIC")
```

```r
# Compare the equivalence classes of both DAGs
all.equal(cpdag(dag1), cpdag(dag2))
```

```r
# Learn structure using HC with different imaginary sample size
dag1 <- hc(asia, score = "bde", iss = 5)
dag2 <- hc(asia, score = "bde", iss = 10)
par(mfrow = c(1, 2), mar = c(1, 1, 2, 1))
graphviz.plot(dag1, main = "ISS = 5")
graphviz.plot(dag2, main = "ISS = 10")
```

```r
# Compare the equivalence classes of both DAGs
all.equal(cpdag(dag1), cpdag(dag2))
```

```r
# Learn structure using HC with random initial structure
set.seed(12345)
dag1 <- hc(asia, score = "bde", start = random.graph(colnames(asia)))
dag2 <- hc(asia, score = "bde", start = random.graph(colnames(asia)))
par(mfrow = c(1, 2), mar = c(1, 1, 2, 1))
graphviz.plot(dag1, main = "Random DAG 1")
graphviz.plot(dag2, main = "Random DAG 2")
```

```r
# Compare the equivalence classes of both DAGs.
all.equal(cpdag(dag1), cpdag(dag2))
```

A.2 Question 2: Learning, fitting and classifying

```r
# Split the data into 80% training and 20% test
n <- dim(asia)[1]
set.seed(12345)
id <- sample(1:n, floor(n * 0.8))
train <- asia[id, ]
test <- asia[-id, ]

# Learn a structure from the training data and plot it against the true DAG
hc_dag <- hc(train, score = "bde", iss = 5)
true_dag <- model2network("[A][S][T|A][L|S][B|S][D|B:E][E|T:L][X|E]")
par(mfrow = c(1, 2), mar = c(1, 1, 2, 1))
graphviz.plot(hc_dag, main = "HC Learned")
graphviz.plot(true_dag, main = "True DAG")

# Fit parameters using maximum likelihood estimation
hc_dag_fit <- bn.fit(hc_dag, data = train, method = "mle")
true_dag_fit <- bn.fit(true_dag, data = train, method = "mle")
```

```r
# Convert both fitted networks to gRain objects for exact inference
hc_grain <- as.grain(hc_dag_fit)
true_grain <- as.grain(true_dag_fit)
```

```r
# Classifies the test set: S = "yes" if P(S = yes | evidence) > 0.5, else "no"
classifier <- function(model, observed_nodes) {
  preds <- numeric(nrow(test))
  for (i in 1:nrow(test)) {
    # Get the states of the current row
    states <- as.matrix(test[i, observed_nodes])
    # Set the evidence of current row
    evidence <- setEvidence(model, nodes = observed_nodes, states = states)
    # Query posterior probability distribution of node S
    pred <- querygrain(evidence, nodes = "S")$S
    preds[i] <- ifelse(pred["yes"] > 0.5, "yes", "no")
  }
  return(preds)
}
```

```r
# Use all nodes except S (column 2) as evidence
nodes <- colnames(test[-2])

# Classify with the network learned by hill climbing
hc_model_preds <- classifier(hc_grain, nodes)
table(test$S, hc_model_preds)

# Classify with the true network
true_model_preds <- classifier(true_grain, nodes)
table(test$S, true_model_preds)
```

A.3 Question 3: Classification with the Markov blanket only

```r
# Restrict the evidence to the Markov blanket of S only
hc_mb <- mb(hc_dag_fit, node = "S")
true_mb <- mb(true_dag_fit, node = "S")

# Classify using the Markov blanket as observed nodes
hc_mb_preds <- classifier(hc_grain, hc_mb)
table(test$S, hc_mb_preds)

true_mb_preds <- classifier(true_grain, true_mb)
table(test$S, true_mb_preds)
```

A.4 Question 4: Naive Bayes classifier

```r
# Naive Bayes classifier assumes that all feature variables are conditionally
# independent given the class variable S.
# This can be represented as a Bayesian network where the class node S is the
# single parent of all feature nodes and no edges exist between features.
nbc_network <- model2network("[S][A|S][T|S][L|S][B|S][E|S][X|S][D|S]")
graphviz.plot(nbc_network, main = "Naive Bayes classifier model")

# Fit the network and convert to a gRain model.
```

```r
nbc_fit <- bn.fit(nbc_network, data = train, method = "mle")
nbc_grain <- as.grain(nbc_fit)

# Classify the test set with the naive Bayes classifier.
nbc_preds <- classifier(nbc_grain, nodes)
table(test$S, nbc_preds)
```